

DYNAHUG: Learning to Detect Malicious Pre-trained Models via Dynamic Analysis

Abstract. Pre-trained machine learning models (PTMs) are commonly provided via Model Hubs (e.g., Hugging Face) in standard formats like Pickles to facilitate accessibility and reuse. However, this ML supply chain setting is susceptible to malicious attacks that are capable of executing arbitrary code on trusted user environments, e.g., during model loading. To detect malicious PTMs, state-of-the-art detectors (e.g., PickleScan) rely on rules, heuristics, or static analysis, but ignore runtime model behaviors. Consequently, they either miss malicious models due to under-approximation (blacklisting) or miscategorize benign models due to over-approximation (static analysis or whitelisting). To address this challenge, we propose a novel technique (DYNAHUG) which detects malicious PTMs by learning the behavior of benign PTMs using dynamic analysis and machine learning (ML). DYNAHUG trains an ML classifier (one-class SVM (OCSVM)) on the runtime behaviours of task-specific benign models. We evaluate DYNAHUG using over 25,000 benign and malicious PTMs from different sources including Hugging Face and MALHUG. We also compare DYNAHUG to several state-of-the-art detectors including static, dynamic and LLM-based detectors. Results show that DYNAHUG is up to 44% more effective than existing baselines in terms of F1-score. Our ablation study demonstrates that our design decisions (dynamic analysis, OCSVM, clustering) contribute positively to DYNAHUG’s effectiveness.

1 Introduction

Pre-trained machine learning models (PTMs) are typically provided via Model Hubs such as Hugging Face (HF) [24], Kaggle [26], GitHub [21], OpenCSG [31], SparkNLP [36] and ModelScope [29]. These PTM hosting platforms ease model accessibility and reuse; thereby supporting a large community of users. Practitioners and companies rely on Model Hubs for the distribution of their PTMs. Similarly, end users rely on third-party vendors to obtain PTMs. For instance, Hugging Face currently hosts over one (1) million ML artifacts and serves over 18.9 million visitors per month [83].

The huge reliance of the PTM ecosystem on third-party Model Hubs raises serious security concerns. Particularly, Model Hubs and end users are vulnerable to malicious actors. Attackers often upload malicious PTMs on Model Hubs to compromise user safety and system security [87]. This increases the risks of end-users executing malicious models in trusted execution environments (e.g., company servers and user’s personal computers) [69–72] [10, 18, 27, 77]. For instance, researchers have found malicious PTMs on Hugging Face that initiate

a reverse shell connection to external servers, potentially allowing to send victim data to attacker-specified IP addresses [69].

To address this concern, malicious PTM detectors and scanners have been developed by Model Hubs, security engineers and researchers. Hosting platforms, such as Hugging Face, employ scanners for the early detection and mitigation of malicious models [56–58]. For instance, Hugging Face’s default security API employs four (4) closed-source scanners using blacklisted imports, virus scanning, static analysis and rulesets [56–58].

Table 1 describes the characteristics of existing PTM scanners/detectors. On the one hand, state-of-the-art detectors employ static analysis, heuristics, or blacklisting to detect malicious PTMs. For instance, PickleScan, ModelScan and HF PickleScan rely on blacklisted imports to detect malicious models [47, 56, 68]. However, these tools are ineffective in detecting previously unseen malicious payloads since blacklists are often non-exhaustive. Table 2 (last (“PyPI”) column) illustrates how importing PyPI modules that support code execution (e.g., `execute` [8]) evades blacklisting methods (e.g., PickleScan and ModelScan [47, 68]). On the other hand, some approaches (e.g., ModelTracer [51]) employ dynamic analysis or blacklisting to detect malicious models. However, these approaches have high false positive rates – they *wrongly* flag benign models as malicious. More importantly, an attacker can easily evade a whitelist or blacklist set of rules, e.g., by using rare, new or previously unseen imports or system calls. Table 2 (second (“Benign”) column) demonstrates how blacklisting methods (e.g., ModelTracer [51]) can lead to false positives.

To address this challenge, we propose an automated method (called DYNAHUG¹) that learns the behavior of benign models using a combination of dynamic analysis and ML. To the best of our knowledge, DYNAHUG is the first technique that automatically learns to detect malicious models via dynamic behavioral analysis. Figure 1 and Figure 6 present the design and algorithm of our DYNAHUG approach. DYNAHUG learns the behaviors of benign models by first clustering models by tasks (e.g., `text-classification`). Secondly, it conducts dynamic analysis of the model in a sandbox and collects system call traces. In this step, PTMs are executed by loading the model and deserializing it. Next, it trains a one-class SVM that learns the behavior of benign models. At inference, when given the system call traces of a PTM under test (PUT), DYNAHUG automatically detects whether it is malicious or benign.

This work makes the following contributions:

1. **DYNAHUG:** We propose an automated method (called DYNAHUG) that learns to detect malicious PTMs by learning the behavior of task-specific clusters of benign models via dynamic program analysis. DYNAHUG provides an automated learning method for detecting malicious PTMs in Model Hubs such as Hugging Face.
2. **Evaluation:** We evaluate DYNAHUG using over 25,000 PTMs. Our experiments demonstrate that DYNAHUG is effective in detecting malicious models with an F1-score of up to 0.9963 across all datasets.

¹ DYNAHUG means “**D**ynamic **H**ugging Face PTM Detector”

Table 1: Details of state-of-the-art detectors versus our approach (DYNAHUG) showing whether the detector “fully” (●), “partially” (◐), or “does not” (○) employ the specified analysis technique.)

Detection Tools	Rule Based	Import Scanning	Blacklist	Whitelist	Heuristic	Dataflow Analysis	Vulnerability Detection	Restricted Loader	Machine Learning	Dynamic Analysis	Open Source
PickleScan [68]	●	●	●	◐	○	○	○	○	○	○	●
ModelScan [47]	●	●	●	○	○	○	○	○	○	○	●
Pickling [48]	●	●	●	◐	○	●	○	○	○	○	●
MALHug [87]	●	●	●	○	●	○	○	○	○	○	○
PickleBall [64]	◐	○	○	○	●	○	○	●	○	○	○
ModelTracer [51]	●	○	○	○	○	○	○	○	○	●	●
Weights-Only [53]	●	○	○	○	○	○	○	●	○	○	●
JFrog (HF) [57]	●	●	●	○	○	○	○	○	○	○	○
Guardian (HF) [58]	●	●	●	○	○	○	◐	○	○	○	○
ClamAV (HF) [56]	○	○	○	○	○	○	●	○	○	○	●
HF PickleScan [56]	○	●	●	◐	○	○	○	○	○	○	○
DYNAHUG	○	○	○	○	○	○	○	○	●	●	●

- State-of-the-art Comparison:** We compare DYNAHUG to six (6) PTM detectors – PickleScan, ModelScan, Fickling, ModelTracer and LLM-based detectors (Llama-3.1-8B-Instruct-tuned and GPT-5.2). Results show that DYNAHUG is up to 44% more effective than the baselines in terms of F1-score.
- Ablation and Sensitivity Studies:** We report ablation studies examining whether our design decisions (e.g., dynamic analysis, etc.) contribute positively to DYNAHUG’s effectiveness and outperform alternative design choices. Additionally, we conduct probing and sensitivity studies examining the impact of task clustering, and training sizes on DYNAHUG’s performance.

2 Overview

2.1 Problem Definition

In this work, we pose the following scientific question: *Given a PTM, how can we automatically detect that it is malicious (or benign)?* Addressing this question is important to ensure the security of trusted execution environments and Model Hubs. Specifically, we aim to develop an automated method that ensures that malicious PTMs are identified during model execution or when uploaded on Model Hubs. We consider a PTM to be *malicious* if it exhibits insecure or unsafe behaviors that are beyond the intended behaviors of typical PTMs. For instance, PTMs that copy user data, sends user data to an unknown IP address or perform remote code execution are considered to be malicious [9, 12]. Thus, a malicious model detector is effective, if it accurately detects genuinely malicious models and does not misclassify benign models. Otherwise, it is ineffective.

2.2 Key Insight

This work proposes DYNAHUG, an automated technique for malicious PTM detection. The main idea of DYNAHUG is to employ dynamic program analysis and ML to learn the behavior of benign PTMs. The *key insight* of our approach

Table 2: Motivating examples showing the performance of DYNAHUG and baselines in detecting malicious PTMs. Disassembled code/trace snippets in **orange** are benign. Code/trace snippets in **red** shows malicious payloads **✓** meaning the malicious PTM was detected by a tool. **✗** mean the model was classified as benign and **✓** indicates a false positive.

	Benign	MALHUG	PyPI
Model Name	llm-stacking/G_learn_depth [15]	josefharush/gpt2-rs [11]	Zollml/dont_download_this [88]
Description	Benign model importing a Tensor class from torch, not part of the standard library [35]	Model found in MalHug [87] containing blacklisted library eval [61]	Model injected with a payload using library execute from PyPI [88]
Disassembled Code	<pre> 110: \x93 STACK_GLOBAL 111: \x94 MEMOIZE (as 7) 112: \x8c SHORT_BINUNICODE 'torch' 119: \x94 MEMOIZE (as 8) 120: \x8c SHORT_BINUNICODE 'Tensor' 128: \x94 MEMOIZE (as 9) 129: \x93 STACK_GLOBAL 130: \x94 MEMOIZE (as 10) </pre>	<pre> 0: \x80 PROTO 2 2: c GLOBAL '__builtin__ eval' 20: q BINPUT 0 22: X BINUNICODE 'exec('\n\nimport urllib.request; exec(urllib.request. urlopen("https:// pastebin.com/raw /svzph7v").read(). decode())\n\n' or dict()') 158: q BINPUT 2 160: R REDUCE </pre>	<pre> 0: \x80 PROTO 2 2: c GLOBAL 'execute both' 16: q BINPUT 0 18: X BINUNICODE 'nc -e /bin/sh 127.0.0.1 4444' 51: q BINPUT 1 53: \x85 TUPLE1 54: q BINPUT 2 56: R REDUCE 57: q BINPUT 3 </pre>
Dynamic Traces	<pre> 104089 newstatat(AT_FDCWD, "/usr/src/app/.venv/lib/ python3.10/site-packages/ torch/tensor.py", {st_mode=S_IFREG 0644, st_size=74282, ...}, 0) = 0 ... 104089 socket(AF_INET6, SOCK_STREAM SOCK_CLOEXEC, IPPROTO_IP) = 3 104089 bind(3, [sa_family=AF_INET6, sin6_port=htons(0), sin6_flowinfo=htonl(0), inet_pton(AF_INET6, "::1", &sin6_addr), sin6_scope_id=0], 28) = 0 ... </pre>	<pre> ... 370730 socket(AF_INET, SOCK_DGRAM SOCK_CLOEXEC SOCK_NONBLOCK, IPPROTO_IP) = 4 370730 setsockopt(4, SOL_IP, IP_RECVERR, [1], 4) = 0 370730 connect(4, sa_family=AF_INET, sin_port=htons(53), sin_addr=inet_addr ("169.254.169.254"), 16) = 0 370730 poll([{fd=4, events=POLLOUT}], 1, 0) = 1 ([fd=4, revents=POLLOUT]) ... </pre>	<pre> ... 410421 close(7) = 0 410421 close(9) = 0 410421 getdents64(5, 0x7ffcfc8560c80 /* 0 entries */ , 280) = 0 410421 close(5) = 0 410421 execve("/bin/sh", ["/bin/sh", "-c", "nc -e /bin/sh 127.0.0.1 4444"], 0x611b65855370 /* 46 vars */ <unfinished ...> 410416 <... vfork resumed> = 410421 ... </pre>
Fickling [48]	✓ (Tagged as unsafe)	✓	✓ (Tagged as unsafe)
PickleScan [68]	✗	✓	✗
ModelScan [47]	✗	✓	✗
HF_JFrog [57]	✗	✓	✗
HF_Guardian [58]	✗	✓	✗
HF_ClamAV [56]	✗	✗	✗
HF_PickleScan [56]	✓ (marked as needing attention)	✓	✓ (marked as needing attention)
ModelTracer [51]	✓	✓	✓
DYNAHUG	✗	✓	✓

is that, for a specific ML task, malicious PTMs often exhibit behaviors that are unique, rare or different from the behaviors of benign models. In particular, we posit that for a specific ML task, (a) benign PTMs exhibit common behaviors, e.g., they often use similar system calls (with similar frequency) and (b) malicious PTMs often exhibit unique, outlier behaviors that are rarely exhibited by benign PTMs. Hence, we hypothesize that automatically learning the behaviors of benign PTMs is applicable for detecting malicious PTMs. Malicious PTMs are known to perform certain operations (e.g., remote code execution) that are never or rarely performed by benign PTMs [87]. This is evident from the fact that malicious payloads require certain system calls for attack orchestration. For instance, malicious PTMs that perform remote code execution often employ system calls (such as `execve`) which are rarely used by benign models since PTMs do not typically execute arbitrary code or require shell access [51]. This insight is evident in the literature; previous works have shown that certain behaviors (e.g., imports or system calls) are security sensitive, unique to malicious models or identifiable as safe, unsafe or sensitive [51, 87].

2.3 Motivating Examples

Table 2 shows examples of real-world benign and malicious PTMs. The two malicious models (last two columns) allow an attacker to conduct dynamic execution of arbitrary Python code during model loading or deserialization. As shown in Table 2 (“MALHUG” third column), most tools detect the malicious payloads involving builtin Python library imports (e.g., `os`). In particular, such imports are often used to orchestrate dynamic code execution in Python. Thus, they are typically added to the blacklist rules of most detectors.

However, most tools are unable to detect uncommon or rare PyPI modules that allow for dynamic code execution (“PyPI” last column). State-of-the-art detectors miss such modules due to the non-exhaustive nature of blacklists. Indeed, there are numerous PyPI libraries that allow for dynamic code execution which makes it impossible to construct a complete blacklist.² Additionally, we show that some detectors (e.g., Fickling) flag almost all models as malicious. This is due to its non-exhaustive set of safe and unsafe opcodes and library imports. Overall, state-of-the-art detectors suffer from under- or over- approximation due to static, incomplete sets of blacklisting or whitelisting rules.

Unlike most detectors, DYNHUG detects the two malicious models in Table 2 and it correctly classifies the benign model. This is due to its use of *task-specific clustering*, *dynamic analysis* and *ML*, which allows it to accurately learn the behavior of benign models and flag the behaviors of malicious models. In DYNHUG, dynamic analysis allows to collect behaviors of benign models, and ML allows to generalize learned behaviors beyond a static rule set. DYNHUG is able to detect malicious PTMs using rare library imports (Table 2 column 1), since such imports use specific system calls which are rarely used by benign models. This demonstrates the importance of dynamic analysis. Our task-specific clustering allows to narrow down the set of behaviors that are unique to a specific category of PTMs. This is inspired by previous works in anomaly detection which have demonstrated that specific categories of applications exhibit similar program behaviors [59, 60, 76].

Note on Traditional malware scanners: Traditional malicious app detectors are not amenable to malicious Pickle/PTM detectors due to the difference in the behavioural profile of PTMs vs. apps or traditional software. For instance, Hugging Face has two main types of traditional malware detectors (ClamAV and VirusTotal), and three specialised PTM scanners. At the time of our Hugging Face analysis, only the specialised scanners detect the malicious PTM datasets collected in this work and reported in previous works (MALHUG). In particular, the three specialised PTM scanners such as PickleScan and JFrog are more effective at detecting malicious PTMs than traditional scanners (see Table 2).

Furthermore, we uploaded five (5) models from MALHUG to VirusTotal [6] to demonstrate the issue. We note that none of the five (5) malicious models are detected [1–4, 7] by any of the 64 security scanners on VirusTotal. These

² The first 20 pages of PyPI when searching “execute” [20] contains over 20 modules that can perform reverse shell execution, many of which are not blacklisted by the state-of-the-art detectors (as at 10 Oct 2025).

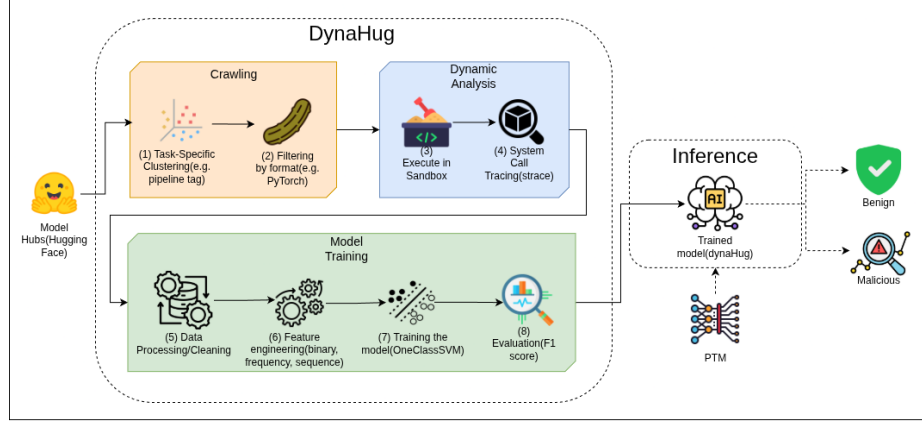


Fig. 1: Workflow of DYNAHUG

64 scanners are from well-known security vendors like Kaspersky, SentinelOne, BitDefender and CrowdStrike. However, this same pickle file is easily detected by pickle security scanning tools like Picklescan [68].

2.4 Novelty vs. State-of-the-art

Table 1 illustrates the novelty of our approach (DYNAHUG) with respect to state-of-the-art techniques. Detectors can be divided into two classes, namely static detectors and dynamic detectors. On one hand, static detectors employ static analysis (e.g., disassembling, dataflow analysis or tainting) alongside blacklisting or whitelisting of specific imports, system calls or opcodes. For instance, PickleScan [68] employs a combination of static analysis, blacklisting and whitelisting of safe and unsafe imports and opcodes for malicious PTM detection (see Table 1). On the other hand, there are few dynamic detectors. Notably, ModelTracer [51] combines dynamic analysis and a blacklist of system calls to detect malicious PTM. However, ModelTracer’s blacklisting limits it from capturing rare or previously unseen imports or system calls and does not allow for subtlety in blacklisted calls. As exemplified in Table 2 (column two), ModelTracer detects the benign model as malicious because of the presence of a socket call, which is part of the blacklist. Additionally, traditional anti-virus (e.g. VirusTotal, ClamAV) are ineffective in detecting malicious PTMs since they are general software vulnerability scanners, they are not specialised for detecting anomalous behaviors in PTMs. This is evident in the poor performance of ClamAV on our motivating examples (Table 2 HF_ClamAV). In particular, anti-virus perform poorly on malicious PTMs due to the difference in the behavioral profile of PTMs versus traditional software. Meanwhile, DYNAHUG is able to correctly classify malicious models as benign since it does not rely on a rulelist. The system level granularity of DYNAHUG’s traces contributes to its performance since the hundreds of imports/opcodes that are manually blacklisted or whitelisted by existing approaches translate to a limited set of system calls during DYNAHUG’s behavioral analysis.

To the best of our knowledge, DYNAHUG is the first approach that employs ML and dynamic analysis for malicious PTM detection. DYNAHUG is unique with its combination of task-specific clustering, dynamic analysis and machine learning. As shown in Table 1, DYNAHUG is the *only* tool that does not rely on blacklisting or whitelisting rules to detect malicious PTMs and the *only* detector that deploys ML for malicious PTM detection. Besides, DYNAHUG is only one of two tools that employs dynamic analysis.

2.5 Threat Model

Attack assumptions: In this work, we assume the attacker modifies an existing model or acts as a third-party model provider supplying a PTM containing arbitrary malicious payloads, e.g., reverse shell. The malicious model is distributed through Model Hubs, provided online or directly sent to users. The attacker provides the malicious model as well as instructions to load/execute the model. This attack setting is practical and fits how models are provided on Model Hubs (Hugging Face [24], GitHub [21] and Kaggle [26]).

Defense assumptions: We assume the defender loads the model following the instructions provided by the model provider. We do not require access to the source code of the model or need security expertise (e.g., to modify how the model is loaded), except the instructions provided by the provider (e.g., in the model card or README). We do not assume access to module libraries or need to modify the default model loading instruction. We assume the model under test is executable in a sandbox and the attacker does not employ anti-debugging techniques. These assumptions are practical, relying on the trust of the victim in the Model Hubs and third-party vendors and does not require additional security or ML expertise or external tool except the model and the model loading instructions. This setting is realistic and similar to the manner end-users obtain PTMs from Model Hubs – Hugging Face [24], GitHub [21] and Kaggle [26].

3 Methodology

Figure 1 illustrates the high-level workflow of DYNAHUG. Figure 6 (Appendix) describes the algorithm for DYNAHUG as detailed in subsection 9.1.

3.1 Detailed Methodology

Crawling and Clustering: The goal of this phase is to collect relevant models from a Model Hub. We make use of the proprietary Model Hub API (e.g., `huggingface_hub` [23]) to filter the repositories based on the tag and the type of the model artifact as shown in Figure 6. Prior works, such as *Gorla et al.* [60], demonstrate that Android applications that are similar, in terms of their descriptions, should also behave similarly. Analogously, our underlying assumption is that models which are assigned the same tags should behave similarly. Task tags in Model Hubs identify the task a particular model is

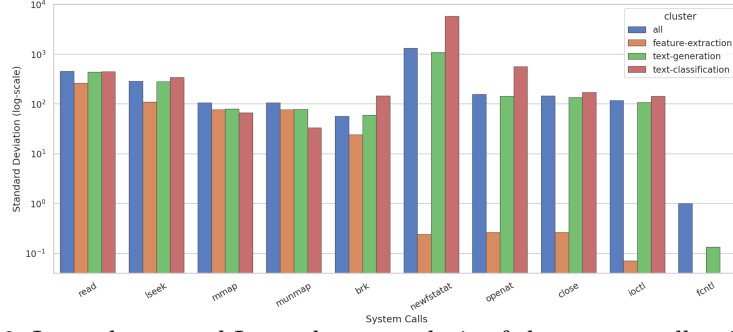


Fig. 2: Inter-cluster and Intra-cluster analysis of the system calls with the maximum difference in mean frequency between clusters.

designed for. For instance, the `text-generation` pipeline tag in HF identifies models that are designed for text generation tasks. This helps narrow down the expected behaviour of a benign model from that particular tag. We also evaluate our task-clustering hypothesis by comparing the standard deviation (SD) of the mean frequency of the top-10 most differing system calls across clusters, using the top-2000 most liked *text-generation*, *text-classification*, and *feature-extraction* from HF. In particular, we examine the SD of the mean frequency of system calls for four (4) clusters, namely (a) within clusters (*intra-cluster*) for text-generation, text-classification and feature-extraction, and (b) without clustering (*inter-cluster*, “all”). Figure 2 shows that there are differences in the SD of the mean frequency within and outside clusters making it imperative to cluster to correctly capture the fine-grained behavioral profile of the benign PTMs and ease malicious PTM detection.

In our analysis of HF, we rank the models according to the likes, and filter models based on pipeline tags with the help of the `huggingface_hub` [23] library in Python. The intuition is that popular models, models which have a higher amount of “likes”, are less likely to be malicious. We focus our efforts on PyTorch models since 95% of all malicious models in HF are known to be PyTorch models [52]. Hence, when retrieving the models, we make sure that a `pytorch_model.bin` is present. We look for this file in particular since it is common convention within Hugging Face (HF) to store the model weights for inference inside `pytorch_model.bin`. This is to ensure that the `transformers` library from HF is able to find the weights file when trying to load a model for inference [25]. Next, the fetched models are stored in a Google Cloud Storage (GCS) Bucket [16] and sent to a sandbox for dynamic analysis.

Dynamic Analysis: DYNAHUG simulates an inference workflow of a user trying to run a model downloaded from a Model Hub. We build a Docker [55] container to load/deserialize the PyTorch models, emulating a reproducible environment similar to that of a user. Docker containers provide OS-level virtualization to isolate any processes running within it from the host machine, preventing any damage from potentially malicious models, e.g., Remote Code Execution (RCE) on the host machine [41]. Moreover, since the output logs from `strace` are non-deterministic and varies with the environment on which the PyTorch model is deserialized and the hardware resources available at that point in execution, it is imperative to have control on the environment, which Docker provides [50].

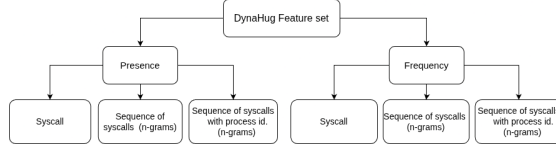


Fig. 3: DYNAHUG’s feature set

```

22928 execve("/usr/bin/cat",
["cat",
"/home/sandbox/.aws/secrets"],
0x5d0f8b34df28 /* 55 vars */
<unfinished ...>
22927 <... vfork resumed> =
22928
22927 rt_sigprocmask(SIG_SETMASK,
[], ~[KILL STOP RTMIN RT_1], 8)
= 0
22928 <... execve resumed> = 0
  
```

Fig. 4: Strace log snippet from deserialization [9]

DYNAHUG provides the flexibility of choosing the runtime observability tools (e.g., `strace` [37], `tcpdump` [39], `eBPF` [19], etc.) to the user depending on the runtime aspects they wish to monitor. Once the runtime observability tool records the deserialisation process of the PyTorch model, the logs are saved for further processing in the upcoming phases.

In our analysis, we utilize `strace`, a diagnostic userspace utility used to monitor interactions between processes and the Linux kernel which includes system calls, signal deliveries and change of process state [37]. `strace` monitors system calls related to File System Management, Networking, Process Management, Memory Management, etc. which covers almost all key operations to give a broad overview of system activity. Previous works [51] also utilize `strace` for dynamic analysis and detection of malicious PTMs. `strace` outputs a detailed raw log file which contains the different system calls and their arguments. For instance, Figure 4 shows the system call, `execve`, running the unix tool, `cat`, to display AWS secrets on the terminal output in a folder commonly known to store them. **Model Training:** To train our classifier effectively, it is important that the input data be represented in a manner that the model can interpret efficiently.

Data Processing: DYNAHUG extracts meaningful information such as system call frequency and stores it in structured formats such as CSV and Apache Parquet.

Feature Engineering: DYNAHUG prepares a presence and frequency feature set for system calls as shown in Figure 3 to feed into the classifier. Presence features indicate the presence of a certain feature with a binary value of either 0 or 1. Frequency features indicate the frequency of occurrence of a certain feature. These features would provide a detailed insight on the statistical distribution of system calls during the deserialization process. To avoid bias towards certain system calls due to their magnitude, DYNAHUG normalizes frequency features to the same scale.

In our analysis, we utilize the `-c` flag in `strace` to get the summarized information of system calls (Section 3.1) during deserialization. The frequency of system calls is parsed from this summarized data and stored in Apache Parquet, since it is faster to load than CSV [74]. DYNAHUG fetches the presence and frequency features for plain system calls and applies normalization with the help of `StandardScaler(nomemean=True)` class in `scikit-learn` [40]. Next, the established feature set is passed to the training component. We have also evaluated other features, e.g., n-grams and Process IDs (see section 4).

Training: DYNAHUG performs hyperparameter tuning while training a one-class SVM. The models trained from each parameter setting are evaluated using F1-score. The best performing model is chosen as DYNAHUG and used during inference. At inference, DYNAHUG follows a similar approach to training to retrieve and process the data from the model analysis (Section 9.1). The processed data is passed into DYNAHUG for detection – "Benign" or "Malicious".

4 Experimental Setup

4.1 Research Questions

- **RQ1 Effectiveness:** How effective is DYNAHUG in detecting malicious PTMs?
- **RQ2 State-of-the-art Comparison:** How does DYNAHUG compare to the SOTA detectors?
- **RQ3 Ablation study:** What are the contributions of our design decision (e.g., static vs. dynamic features) to the performance of DYNAHUG?
- **RQ4: Probing and Sensitivity Study:** What is the impact of clustering on DYNAHUG’s performance? How sensitive is DYNAHUG to different training dataset sizes?

4.2 Datasets

Table 3: PTMs on Top six (6) Model Hubs.

Hub	#Models	Formats	Text Generation	Text Classification	Feature Extraction
Hugging Face [24]	1881296	Pytorch, Tflow, gguf	296411	98414	13864
GitHub [21]	150685	Pytorch, Tflow, gguf	1732	4427	1957
Spark NLP [36]	135372	Custom	25181	35248	N/A
OpenCSG [31]	111189	Pytorch, Tflow, gguf	4224	278	484
ModelScope [29]	80574	Pytorch, Tflow, gguf	20307	1172	740
Kaggle [26]	3140	Pytorch, Tflow, gguf	800	197	N/A

In our experiments, we employ Hugging Face (HF) as our Model Hub, since it is the most popular model hub with the largest set of models. Table 12 provides details of the top six

model hubs. We collect our dataset of benign models from the Top-K (default 2000) models on HF. Our experiments involve three (3) task categories from HF, namely *text-generation*, *text-classification* and *feature-extraction*.

Table 4: Details of training and evaluation datasets across all clusters.

Datasets	Range (Sorted By Most Likes)	Benign				Malicious				Total
		Text Gen	Text Class	Feature Extraction	No Cluster	Text Gen	Text Class	Feature Extraction	No Cluster	
Real (HF)	$(1 - 3000) \cup (4601 - 5625)$	5025	5001	5001	3000	3	0	0	0	18030
Real (MALHug)	-	0	0	0	0	20	2	17	0	39
Real (PickleBall)	-	0	3	16	0	2	2	0	0	23
Synthetic (MALHug Injected)	3600-4600	0	0	0	0	1000	1000	1000	1000	4008
Synthetic (PyPI Injected)	3600-4600	0	0	0	0	1000	1000	1000	0	3008
Real (Total)	-	5025	5004	5017	3000	25	4	17	0	18092
Synthetic (Total)	-	0	0	0	0	2000	2000	2000	1000	7016

Table 4 describes our collected dataset for each cluster. For the text-generation cluster, we first filter out models that are flagged by HF as malicious and that are not part of known malicious PTMs [64, 87]. In the top 3,000 models, we found and filtered out two (2) PTMs. Specifically, we download the top 3000 models ranked by most likes, from which we filtered out two (2) models that are flagged

by HF and MALHUG [87] as malicious. We download another 1000 benign models so that we can evaluate DYNAHUG on them, and also inject them with malicious payloads to make our injected set of 2000 models. To leave an appropriate gap between the training set and the injection set to avoid contamination, we collect our next set of 1000 models from the 3600 to 4600 top liked models. Furthermore, we download the next 1025 models that are not flagged by HF as an evaluation set to test the model against, and balance our dataset with regards to our malicious set. The above setting describes all text-generation experiments (**RQ1** to **RQ4**). Then, we repeat the steps described above for text-classification and feature-extraction (**RQ1**) and non-clustered experiments (**RQ4**).

4.3 Injected malicious models

To ensure detectors generalise to unseen models and address class imbalance, we augment our evaluation dataset by injecting malicious payloads into benign models using models from HF and malicious payloads from MALHUG and libraries from PyPI that allow for code execution.

MALHUG injected models: Using Fickling [48], we automatically inject 64 malicious payloads from MALHUG into 1000 benign models from HF. [Example 1.1](#) shows an example of a MALHUG injected payload. We ensure a model is valid by checking that it is successfully loaded by PyTorch with `torch.load()`. Injection process times out and proceeds to the next payload when there is a blocking operation from a MALHUG payload, e.g., it requires user input (`input()`).

PyPI injected models: The goal of the PyPI injected model is to provide more realistic malicious payloads (e.g., used for information reconnaissance, or reverse shells), since most MALHUG payloads are Proof-Of-Concepts (PoCs). Hence, the payloads were chosen from well-known external sources such as PayloadsAllTheThings [85] and revshells.com [45]. We automatically inject 20 of these malicious payloads using PyPI libraries [79] into 1000 benign models from HF. [Example 1.2](#) shows an example of our PyPI injected payload. These libraries allows the user to either execute system commands (similar to `os.system()`), or execute Python code (similar to `exec()`). Using these 20 malicious pickles, we directly inject their Pickle bytecode into the 1000 benign models we collected, after modifying the required stack addresses such that there is no memory conflict between the original model and the injected payload. For each injected payload, we setup a test oracle to confirm that the resulting models are valid, and injection was successful. To account for the blocking operations of the reverse shells, we start a netcat [73] server that sends an `exit` instruction to terminate the reverse shell.

4.4 Training Process

Architecture Selection: We begin the training process after collecting the required number of benign models from crawling through HF as detailed in [Section 9.2](#) (Appendix). The one-class SVM (OCSVM) was selected since it is

a popular choice in unsupervised learning based anomaly detection, alongside Stochastic Gradient Descent one-class SVM (SGDSVM) and Isolation Forest (IF). These architectures require only one-class of training data, which becomes essential in the case of a dataset imbalance. ML algorithms supporting one-class training are necessary in the PTM setting since there are millions of benign models but few malicious models.

Feature Engineering: Figure 3 shows the variety of features that were used in the experiments. When constructing features related to individual system calls, a platform-specific exhaustive list of system calls was used. This was adopted to prevent Out Of Vocabulary issues that could arise from unseen system calls in the test set. The exhaustive list was obtained from the Linux kernel source code in the docker container [38], corresponding to the x86-64 architecture.

DYNHUG has three (3) different features sets for training. First, *presence and frequency* features capture the presence and frequency of individual system calls (e.g., “execve”, “read”, “write”). *Sequence* based features represent the presence and frequency of system call sequences (n-grams) (e.g., “execve:read”, “read:write”). *Sequence* features are represented with 2-grams, as this configuration offers the best trade off between model complexity and performance as shown in Table 13. Although higher order n-grams (i.e., 3-gram, 4-gram) provide a better spacial understanding of system calls to the model, preliminary analysis showed that choosing these n-gram configurations would lead to poorer performance.

Finally, *Process Sequence* based features incorporated presence and frequency of process ID (PID) system call sequences (e.g., “P1:close_P5:rt_sigprocmask”, “P4:rseq_P1:read”) . The raw PID value was not used to represent the process since each process has a unique PID in every run which would introduce noise into our training dataset. Hence, a generalized form of the PID is used where the parent process is P1 and any subsequent processes would be numbered accordingly. Process sequence features are also 2-grams.

Dataset Split: Datasets are split into train, validation and test sets in the ratio 80:10:10. The training dataset consists exclusively of the benign models collected from HF. However, for the validation and test set, we combined an equal number of benign and malicious models. Specifically, the malicious models were randomly sampled from MALHUG and MALHUG-injected from the chosen task cluster. This makes it easier to facilitate a more reliable assessment on the model’s ability to distinguish between benign and malicious models. The malicious models in the validation or test set contains half of MALHUG from the selected task cluster, and the rest are supplemented by MALHUG-injected to solve the dataset imbalance between benign and malicious models in the validation and test set. For instance, our default setting consists of 2000 models from the *text-generation* cluster. The size of validation and test set is 200 (10% of 2000). MALHUG has 25 malicious models belonging to the *text-generation* cluster. Hence, in the split, half of these models (12) would go into the validation and the other half (13) would be part of the test set. To avoid data imbalance between benign and malicious, the rest of the models (188 or 187 for validation or test set) are randomly sampled from MALHUG-injected.

Hyperparameter Tuning Strategy: DYNAHUG performs Grid Search on a set of hyperparameters which are tailored to a specific classifier architecture. We employ a 5-fold cross validation procedure to ensure that we get the average F1-score for the best model evaluation.

Hyperparameter Choices: For OCSVM, we tune parameters *nu* (0.01 to 0.5), *kernel* (linear, RBF and sigmoid), and *gamma* (0.01, 0.1, 1 and auto). For the SGDSVM, we additionally tune *batch_size* (32, 64, 128 and 1000). Finally, for IF, we tune *n_estimators* (50 to 150), *contamination_rate* (0.001, 0.01, 0.1 and auto) and *max_samples* (128, 256, 512 and 1000).

Data Preprocessing: Once the hyperparameter combination is selected, we utilize the DictVectorizer from scikit-learn [40] to transform the feature-value mapping to a matrix which is comprehensible to the classifier. The values within this matrix are normalized by applying the StandardScaler (normean=True) class. The centralization of the values by the mean is disabled to prevent the data from losing its sparse nature and optimizing memory usage by avoiding the sparse matrix from becoming a dense matrix. This normalized data is fitted to the classifier of choice and this process is repeated until a classifier with the highest possible average F1-score on the validation set is selected as the final trained model. The final trained model is also evaluated on the test set to gain deeper insights of the model’s performance on unseen data.

4.5 Metrics and Measures

DYNAHUG uses accuracy, precision, recall and F1-score to evaluate trained models. The F1-score was utilized as a judging criteria for selecting the best model during hyperparameter tuning. The F1-score is the harmonic mean between precision and recall. This means that an increase in False Positives (affects Precision) or False Negatives (affects Recall) would punish the F1-score, promoting a balance between False Positives and False Negatives.

4.6 Research Protocol

Firstly, we evaluate DYNAHUG using three (3) different tasks (**RQ1**) by training our classifier on the text-classification, text-generation and feature-extraction cluster. The goal is to measure the effectiveness of our approach on different clusters. Besides, we compare DYNAHUG to the current state-of-the-art (**RQ2**), we evaluate existing open-source detectors encompassing both static and dynamic analysis. We also evaluate whether LLMs can serve as an effective malicious PTM detector by replacing our classifier with an LLM and using the traces collected from dynamic analysis as an input. We excluded closed-source scanners and Restricted-Loading Environments (RLE) such as Weights-Only Unpickler [53] and PickleBall [64] because they do not scale to our evaluation setting (25K PTMs). Moreover, RLEs make incorrect assumptions about the user and could be easily bypassed. We have detailed our selection criteria and baseline setting in Section 9.3 (Appendix) and Section 9.4 (Appendix).

Next, we conducted an ablation study (**RQ3**) examining our design choices. We examined the contribution of (a) dynamic analysis versus alternatives (i.e., static, dynamic or hybrid), (b) ML architecture (i.e., OCSVM, SGDSVM, and IF) and (c) feature set provided to the classifier for training and inference (i.e., *presence, frequency, sequence, process sequence*). Additionally, we investigate the performance of DYNAHUG with and without task clustering (**RQ4**). Finally, we performed sensitivity analysis (**RQ4**) of the classifier against varying training dataset size, i.e., top 1000, 2000 and 3000 repositories from HF. The settings for ablation, probing and sensitivity study is elaborated in Section 9.5 (Appendix).

4.7 Implementation Details and Platform

DYNAHUG was implemented in about 4.1k lines of Python code. Experiments and data analysis were implemented in 6k lines of Python code. For crawling, we used the Python SDK for hugging face, i.e., `huggingface_hub` [23]. The downloaded models were then archived for future reference inside a GCS Standard Bucket with the help of the `google-cloud-storage` [33] library. For dynamic analysis, the code was containerized using Docker [55]. The PyTorch files were deserialized using `torch` [34] and `strace` [37] was used to observe the runtime behaviour. For the model training phase, NumPy [30] and pandas [32] libraries were utilized for data analysis, processing and manipulation. Steps related to model training and evaluation were done with the `scikit-learn` [40] machine learning library. SHAP [66] was used to explain the impact of the features on which the classifier was trained on. Experiments were setup on n2-highmem-4 (4 vCPUs, 32 GB Memory) Google Cloud Compute Engine instance with Ubuntu, 22.04 LTS Minimal installed. Within this VM, a docker container with a base image of `python:3.10.12-slim` was setup with all project files cloned from the GitHub repository and the required dependencies installed. The LLM baseline experiment was run on Google Colab [17] with a NVIDIA-A100 GPU [75]. HF `transformers` library was utilized to run inference on Llama-3.1. GPT-5.2 was executed through the official OpenAI API via the `openai` library. We provide DYNAHUG’s source code and our experimental data online to support reproducibility and reuse: <https://dynahug-detector.github.io>

5 Results

RQ1: Effectiveness. We found that *DYNAHUG is effective in detecting malicious PTMs across all three (3) clusters*. This is illustrated in Table 5. Specifically, our approach detects 92.8%-100% of real malicious models and 98.07%-99.7% of synthetic injected models across all three (3) clusters. We observed that DYNAHUG generally performs well on the three (3) task clusters across all performance metrics. However, DYNAHUG has a slightly better performance on the text generation cluster than the text classification cluster (0.9963 vs. 0.9893) and the feature extraction cluster (0.9963 vs. 0.9930). Across all metrics, DYNAHUG’s performance is consistently high (between 0.9861 and 0.9995). All in

Table 5: DYNAHUG’s performance on three (3) task-based clusters

Cluster	Benign		Malicious		Overall Performance					
	Real	Injected	Real	Injected	TP	TN	FP	FN	Precision	Recall F1-score
text-generation	1/2025	25/25	1986/2000		2011/2025	2024/2025	1/2025	14/2025	0.9995	0.9931 0.9963
text-classification	28/2004	4/4	1985/2000		1989/2004	1976/2004	28/2004	15/2004	0.9861	0.9925 0.9893
feature-extraction	18/2017	13/17	1994/2000		2007/2017	1999/2017	18/2017	10/2017	0.9911	0.9950 0.9930

Table 6: Comparison of DYNAHUG vs. open-source SOTA’s performance.

Analysis Type	Detector	Benign		Malicious (2025)			Overall Performance					
		HF (2025)	Real	MALHUG	PyPI	TP	TN	FP	FN	Precision	Recall	F1-score
Static	PickleScan [68]	0	23	1000	44	1067	2025	0	958	1	0.5269	0.6902
	ModelScan [47]	0	23	1000	44	1067	2025	0	958	1	0.5269	0.6902
	Fickling [48]	2025	25	1000	1000	2025	0	2025	0	0.5	1	0.6667
Dynamic	ModelTracer [51]	0	25	828	907	1760	2025	0	265	1	0.8691	0.9299
Dynamic	Llama-3.1 [67]	1337	22	993	995	2010	688	1337	15	0.6005	0.9926	0.7483
+ LLM	GPT-5.2 [46]	12	20	988	796	1804	2013	12	221	0.9933	0.8909	0.9393
Dynamic	DYNAHUG (default)	1	25	1000	986	2011	2024	1	14	0.9995	0.9931	0.9963

all, DYNAHUG has a good performance, which is consistently above 0.986 across all metrics and settings. This result implies that the ML-based dynamic analysis technique of DYNAHUG is effective in malicious PTM detection. In summary, our approach (DYNAHUG) generalizes to two different PTM tasks.

DYNAHUG is effective in malicious PTM detection across all three (3) clusters (up to 0.9963 F1-score).

RQ2: State-of-the-art Comparison. We found that *DYNAHUG is up to 44% more effective than the state-of-the-art detectors in terms of F1-score. It outperforms the other baselines by up to 25%, 20.77% and 19.2% in detecting Real Malicious, MALHUG injected and PyPI injected respectively.* Table 6 shows that DYNAHUG outperforms the closest competitor (GPT-5.2) by about 5% (0.9963 vs. 0.9393 F1-score). GPT-5.2 excels in detecting the MALHUG injected set as compared to Llama-3.1 and ModelTracer. ModelTracer is the best non-LLM detection in the baselines. On the one hand, ModelTracer detects all real malicious PTMs, but it does not generalize on the exact malicious payloads when injected in different models. This demonstrates that its detection is not generalizable to new models. In addition, ModelTracer does not perform well on PyPI injected modules, which underscores its non-exhaustive trace blacklisting. On the other hand, the static detectors have the worst performance, followed by the Llama-3.1-8B-Instruct-tuned. DYNAHUG is 44% (0.9963 vs. 0.6902 F1-score) more effective than the best static detectors, i.e., PickleScan and ModelScan. We attribute the poor performance of the static detectors to the lack of behavioral information, use of rule sets and the over-approximation of static analysis. Finally, DYNAHUG is 33% (0.9963 vs. 0.7483) more effective than Llama-3.1 in malicious PTM detection. We believe the poor performance of Llama-3.1 is due to the lack of PTM-specific security knowledge. Overall, this experiment demonstrates the superiority of DYNAHUG, and the importance of its behavioral analysis and learning approach to malicious PTM detection.

DYNAHUG is up to 44% more effective than the state-of-the-art detectors in terms of F1-score.

Table 7: DYNAHUG’s performance under different analysis types (best results are in **bold text**)

Analysis Type	Detector	Benign			Malicious			Overall Performance						
		HF (2025)	Real (25)	Injected (2000)	TP	TN	FP	FN	Precision	Recall	F1-score			
Static	DYNAHUG	2025	25	2000	2025	0	2025	0	0.5000	1.0000	0.6667			
Dynamic	DYNAHUG (default)	1	25	1986	2011	2024	1	14	0.9995	0.9931	0.9963			
Hybrid	DYNAHUG	1	25	1986	2011	2024	1	14	0.9995	0.9931	0.9963			

Table 8: DYNAHUG’s effectiveness with varying feature sets showing ‘freq’ = frequency, ‘seq’ = sequence and ‘proc’ = process (best results are in **bold text**)

Feature Set	Detector	Benign			Malicious			Overall Performance						
		HF (2025)	Real (25)	Injected (2000)	TP	TN	FP	FN	Precision	Recall	F1-score			
presence	DYNAHUG	0	22	1664	1686	2025	0	339	1.0000	0.8326	0.9086			
freq	DYNAHUG	1	25	1986	2011	2024	1	14	0.9995	0.9931	0.9963			
presence, freq	DYNAHUG (default)	1	25	1986	2011	2024	1	14	0.9995	0.9931	0.9963			
presence, freq, seq	DYNAHUG	33	24	2000	2024	1992	33	1	0.9840	0.9995	0.9917			
presence, freq, seq, proc seq	DYNAHUG	51	24	2000	2024	1974	51	1	0.9754	0.9995	0.9873			

RQ3: Ablation and Probing study (Analysis Type). We found that *DYNAHUG with dynamic analysis is more effective than DYNAHUG with static analysis*. Table 7 shows that dynamic analysis in DYNAHUG is twice (2x) as effective as static analysis (precision of 0.5 vs. 0.9995). Meanwhile, hybrid analysis has the same performance as dynamic analysis. We observe that static analysis is not as effective as dynamic analysis. Besides, combining dynamic and static analyses does not improve the performance of DYNAHUG, it is as effective as using *only* dynamic analysis (see Table 7). We attribute the weak performance of static analysis to the lack of precise execution information which causes over-approximation. This result also aligns with the performance of the static detectors in RQ2 (see Table 6). Overall, this result shows that DYNAHUG’s use of dynamic analysis contributes positively to its detection effectiveness.

Dynamic analysis contributes positively to detection effectiveness of DYNAHUG; it is twice as effective as using only static analysis.

Feature Selection: Results show that *the default feature setting of DYNAHUG (presence and frequency of system calls) contributes positively to its effectiveness and outperforms alternative features setting*. More importantly, Table 8 shows that the *frequency of system calls* improves DYNAHUG’s detection effectiveness more than all other feature settings. In comparison to presence only features, the frequency of system calls improves the recall and F1-score of DYNAHUG by up to 19% (0.8326 vs. 0.9931) and $\approx 10\%$ (0.9086 vs. 0.9963), respectively. Meanwhile, employing a larger feature set than the default settings (e.g., adding sequence of system calls and/or sequence of system processes) leads to poorer performance with a reduced the precision

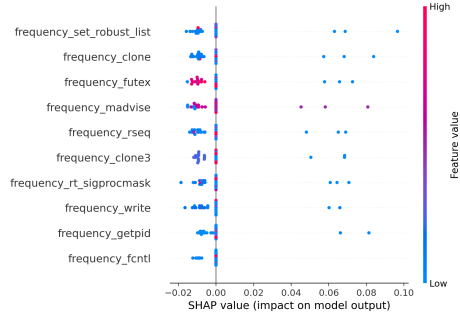


Fig. 5: Top-10 most important features that explain DYNAHUG’s predictions on malicious models

Table 9: DYNAHUG’s performance across different architectures (best results are in **bold** text)

Architecture	Detector	Benign	Malicious		Overall Performance							
		HF (2025)	Real (25)	Injected (2000)	TP	TN	FP	FN	Precision	Recall	F1-score	
IsolationForest	DYNAHUG	4	24	1782	1806	2021	4	219	0.9978	0.8919	0.9419	
OneClassSVM	DYNAHUG (default)	1	25	1986	2011	2024	1	14	0.9995	0.9931	0.9963	
SGDOneClassSVM	DYNAHUG	67	25	2000	2025	1958	67	0	0.9680	1.000	0.9837	

and F1-score (see Table 8). We attribute the performance of DYNAHUG’s *default* setting to its use of the *frequency of system calls* which makes DYNAHUG detect outlier system call distribution. This observation aligns with our explainability findings. For instance, Figure 5 shows that the frequency of `set_robust_list`, `clone` and `futex` calls are the most important features for predicting malicious models. Similarly, the frequency of `futex`, `brk` and `read` are the most important features for predicting benign models as shown in Figure 7 (Appendix). These results show that DYNAHUG’s feature setting contributes positively to its detection effectiveness.

The default feature setting of DYNAHUG outperforms alternative feature settings and the frequency of system calls contributes the most to its performance.

ML Architecture: We observed that *the default model architecture in DYNAHUG (OCSVM) outperforms the tested alternative architectures (IF and SGDOCSVM)*. Table 9 shows that default DYNAHUG (OCSVM) performs best (F1 = 0.9963), followed by DYNAHUG with SGDOCSVM (F1 = 0.9419), then DYNAHUG with IF (F1 = 0.9837). However, we observed that DYNAHUG with SGDOCSVM has a slightly better recall (1.0 vs. 0.9931). We attribute the better performance of the the default DYNAHUG (OCSVM) to the size of our dataset, since the tested alternatives (IF and SGDOCSVM) are often sensitive to the size of the training dataset and more suitable for training larger datasets [78, 80, 81]. In particular, OCSVM is more suitable for DYNAHUG due to DYNAHUG’s task clustering step (causing a reduced training size) and the huge computational cost of training larger datasets. This result demonstrates that the OCSVM architecture fits DYNAHUG’s design and outperforms close alternatives (IF or SGDSVM).

DYNAHUG’s architecture (OCSVM) fits its design and outperforms alternatives (IF or SGDSVM).

RQ4: Probing and Sensitivity Study

Task Clustering: We found that *DYNAHUG with clustering outperforms DYNAHUG without clustering*. Table 10 shows that DYNAHUG performs better when it is trained on individual task-based clusters (e.g., `text-generation`) versus on the Top-2000 models on HF (without clustering). This result holds across all metrics. For instance, the F1-score of the DYNAHUG trained on the text generation cluster is 0.9851, while the non-clustered DYNAHUG has an F1-score of 0.9754. We attribute the performance of the clustered setting to the fact that it learns the specific behavior of a particular task better than the non-clustered

Table 10: Impact of clustering on DYNAHUG (best results are in **bold** text)

Cluster	Detector	Benign Malicious		Overall Performance						
		Set	Set	TP	TN	FP	FN	Precision	Recall	F1-score
text-generation	DYNAHUG	4/200	198/200	198/200	196/200	4/200	2/200	0.9802	0.9900	0.9851
text-classification	DYNAHUG	7/200	199/200	199/200	193/200	7/200	1/200	0.9660	0.9950	0.9803
feature-extraction	DYNAHUG	5/200	197/200	197/200	195/200	5/200	3/200	0.9752	0.985	0.9801
non-clustered	DYNAHUG	9/200	199/200	199/200	191/200	9/200	1/200	0.9567	0.9950	0.9754

Table 11: DYNAHUG’s sensitivity to varying dataset size (best results are in **bold** text)

Training Size	Detector	Benign Malicious		Overall Performance						
		Set	Set	TP	TN	FP	FN	Precision	Recall	F1-score
1000	DYNAHUG	4/100	100/100	100/100	96/100	4/100	0/100	0.9615	1.0000	0.9804
2000	DYNAHUG (default)	4/200	198/200	198/200	196/200	4/200	2/200	0.9802	0.9900	0.9851
3000	DYNAHUG	4/300	297/300	297/300	296/300	4/300	3/300	0.9867	0.9900	0.9884

model. This finding demonstrates the importance of clustering and motivates our design decision to cluster before training.

Task-based clustering contributes positively to DYNAHUG’s performance.

Sensitivity to training dataset size: We found that *the effectiveness of DYNAHUG slightly improves as the training dataset increases*. Table 11 shows that training on a smaller dataset (1000) has a lower F1-score than training on the default DYNAHUG setting (2000), and training on even more PTMs (3000) outperforms the default DYNAHUG setting. DYNAHUG’s performance is sensitive to the size of the dataset. We attribute the performance of larger datasets to the contribution of additional data points for generalizing the training. However, there is a trade-off between analysis cost and improved performance. Thus, we employ the 2,000 training data size for all other experiments, since the performance improvement with 3,000 PTMs is not significant (0.9884 vs. 0.9851) and it is computationally expensive to collect dynamic analysis on a larger set of models across each study, (e.g., thousands of models for additional settings in the ablation studies (RQ3)).

DYNAHUG’s effectiveness slightly improves as the size of the training dataset increases.

6 Limitations and Threats to Validity

Internal Validity: The main threats to internal validity of this work and DYNAHUG are the correctness of our implementation and the reproducibility of our results. To mitigate this threat, we have tested our implementation and resulting models under varying settings to ensure reproducibility (*see RQ3*). We also provide our source code and experimental data to support scrutiny and reuse.

External Validity: The main risk to external validity is the generalizability of DYNAHUG and our findings. In particular, our findings may not generalize to other Model Hubs, tasks or unpopular (non-Top-K) models. To mitigate this

threat, we have performed our experiments using the most popular model hub (Hugging Face), tested on two popular clusters and thousands of benign and malicious models. We have also tested on different parameter settings in our ablation and sensitivity study (**RQ3** and **RQ4**). In addition, to address concerns that the injected payloads may not be representative of real world malware, we have conducted experiments to test our approach on advanced payloads (e.g., Anti-VM detection techniques, staged payloads, obfuscated payloads, and delayed execution payloads) as detailed in Section 9.6 (Appendix).

However, despite best efforts, we acknowledge that our findings may not generalize to other settings, beyond the tested setting.

Construct Validity: To avoid experimenter bias in this work, we have employed thousands of models from Hugging Face and related scanners (e.g., MALHUG). We have also compared our approach to five (5) open-source state-of-the-art detectors and filtered our training set using the scanners provided by Hugging Face security API. Finally, we have also manually inspected the real-world malicious PTMs to confirm they are malicious and constructed test oracles to check that our injector indeed inject malicious payloads.

Additional Dynamic features: The dynamic features explored in this work are not exhaustive. Employing additional features which may improve the performance of DYNAHUG. In future work, we plan to explore additional features such as API call graph, control flow graph, etc. [84, 86].

Ethical Considerations: DYNAHUG is computationally less intensive than using LLMs because our approach relies on classical ML algorithms with 30+ parameters and requires smaller datasets (25K PTM traces), as compared to the billions or trillions of parameters and trillions of tokens of publicly available training datasets in LLMs [5, 14, 65]. Hence, our approach is more sustainable and environmentally friendly compared to inferencing, training or fine-tuning LLMs.

For security reasons, we do not publicly provide all of our injected malicious PTMs or make them available on any platforms, except as an artefact for this paper’s evaluation. We plan to provide the PTMs as an artefact for research purposes after paper acceptance. In the artefact, we will include cautions, disclosure and disclaimer. In particular, we will provide the injected models and a description of the associated libraries for research purposes. In our study, we tested five (5) malicious PTMs on Hugging Face as a Proof-of-concept (PoC) to report the performance of Hugging Face scanners on our PyPI-injected models. These models contain PoC payloads that only demonstrate the capability of the attack without actually carrying out a real attack, e.g., (a) reverse shell to a local IP, rather than an external IP, and (b) printing the list of files in a directory “ls”, rather than more serious shell commands like deleting directories “rm”. We created an HF repository with some PoCs, and explicitly caution against downloading them. The repository’s [88] card README contains a warning that it is intended for security research purposes and should not be downloaded: “Please do not download these models, as they are for research purposes only and may be damaging to your system. Not safe.”

7 Related Works

Static Detectors: These tools aim to detect malicious PTMs by inspecting its code for potential malicious behaviors, e.g., using disassemblers like pickle-tools [64] or Fickling [64]. Detection is typically performed through a blacklist of unsafe opcodes (e.g., PickleScan, Modelscan, HF PickleScan [47, 56, 68]) or a whitelist of safe opcodes (e.g., PickleBall [51]). To further overcome the limits of a static list of rules, some static detectors also employ methods such as heuristics (MALHUG [87]), policy generation (PickleBall [64]) or data flow analysis (Fickling [48]). Unlike these approaches, DYNAHUG employs dynamic behavior of PTMs rather than static analysis and relies on ML rather than static blacklisting or whitelisting.

Dynamic Detectors: Similar to DYNAHUG, ModelTracer [51] detects malicious PTMs using dynamic analysis, in particular, strace [37] and Python’s sys [54]. It processes the generated syscall data and then checks for sensitive syscall presence like exec, connect and chmod as indicators of malicious behaviour. Invariably, it employs a blacklist of system calls to detect malicious PTMs. Unlike ModelTracer, DYNAHUG relies on the ML classifier to learn the behaviours of benign models, rather than relying on a static rule list which can be easily bypassed. and require expert review.

8 Conclusion

This work proposes a novel method for detecting malicious PTM models. The goal of our work is to ensure that PTM users can detect malicious PTM models provided by third-party vendors in the ML supply chain system. In particular, we aim to ensure the safety of PTMs uploaded on Model Hubs like Hugging Face or executed in trusted user environments. Our approach (DYNAHUG) employs a combination of dynamic analysis, task-specific clustering and ML to detect malicious PTM models. We orchestrate DYNAHUG by employing a one-class SVM to learn the behavior of the top-K (2000) most liked models in a task-specific cluster (e.g., text generation) of Hugging Face. We also evaluate DYNAHUG using over 25,000 benign and malicious models from different sources, including Hugging Face, MALHUG and injected malicious payloads using MALHUG and PyPI modules. Besides, we compare the effectiveness of DYNAHUG to five baseline detectors, including static, dynamic and LLM-based detectors. Our evaluation results demonstrate that DYNAHUG is effective in detecting malicious PTMs, outperforms the baselines. This work motivates the need to employ behavioral analysis for PTM detection and reduce reliance on blacklists or whitelists. In the future, we aim to study additional model hubs and task clusters, and deploy DYNAHUG in the wild. To support reproducibility and reuse, we provide DYNAHUG’s source code and our experimental data: <https://dynahug-detector.github.io/>

References

1. admko/evil_test - virustotal. <https://www.virustotal.com/gui/file/e7191fdaf9bc9b19266e8bcf348788a4493cdbe03baadf67a483b7093783aa4b>, [Accessed 16-04-2026]
2. drhyrum/bert-tiny-torch-picklebomb-virustotal. <https://www.virustotal.com/gui/file/c8354611d7d2f2437aa2ce1af05760ae2a4acbf21e2b3a4cbe77024130bf8df>, [Accessed 16-04-2026]
3. Farishijazi/totally-harmless-model - virustotal. <https://www.virustotal.com/gui/file/65a011d547d25bd600580861f8d47447b6fedc4fb11d4c01ed348ef58e510b3b>, [Accessed 16-04-2026]
4. Narsil/totallysafe - virustotal. <https://www.virustotal.com/gui/file/5949e26161da6e53cf2771aa7de4212b70d5aa065baab49e600bb295f11fad11>, [Accessed 16-04-2026]
5. Open-sourced training datasets for large language models (llms). <https://kili-technology.com/blog/9-open-sourced-datasets-for-training-large-language-models>, [Accessed 16-04-2026]
6. Virustotal - virustotal.com. <https://www.virustotal.com/gui/home/upload>, [Accessed 16-04-2026]
7. zp Brent/reuse - virustotal. <https://www.virustotal.com/gui/file/2337248d2e27e905e483ce8d1740035a98fd915d01c575fb5235d60016b8a56a>, [Accessed 16-04-2026]
8. Execute – pypi.org. <https://pypi.org/project/execute/> (2010), [Accessed 10-11-2025]
9. ankushvangari-org2/unsafe_model · Hugging Face. https://huggingface.co/ankushvangari-org2/unsafe_model (2023), [Accessed 09-11-2025]
10. Federal register :: Request access. <https://www.federalregister.gov/documents/2023/11/01/2023-24283/safe-secure-and-trustworthy-development-and-use-of-artificial-intelligence> (2023), [Accessed 09-11-2025]
11. jossefharush/gpt2-rs at main — huggingface.co. <https://huggingface.co/jossefharush/gpt2-rs/tree/main> (2023), [Accessed 28-01-2026]
12. star23/round2. <https://huggingface.co/star23/round2/tree/main> (2023), [Accessed 09-11-2025]
13. gabejabe/bsidesSF-gordon-ramsey · Hugging Face — huggingface.co. <https://huggingface.co/gabejabe/bsidesSF-gordon-ramsey> (2024), [Accessed 11-11-2025]
14. Llama 3.1 release. <https://ai.meta.com/blog/meta-llama-3-1/> (2024), [Accessed 09-11-2025]
15. llm-stacking/G_learn_depth at main — huggingface.co. https://huggingface.co/llm-stacking/G_learn_depth/tree/main (2024), [Accessed 11-11-2025]
16. Cloud Storage. <https://cloud.google.com/storage?hl=en> (2025), [Accessed 11-11-2025]
17. colab.google. <https://colab.google/> (2025), [Accessed 14-11-2025]
18. cve.org. <https://www.cve.org/CVERecord/SearchResults?query=pickle> (2025), [Accessed 09-11-2025]
19. ebpf - introduction, tutorials & community resources — ebpf.io. <https://ebpf.io/> (2025), [Accessed 12-10-2025]
20. Execute Search — pypi.org. <https://pypi.org/search/?q=execute> (2025), [Accessed 11-11-2025]

21. GitHub. <https://github.com/> (2025), [Accessed 07-11-2025]
22. GitHub - s2e-lab/hf-model-analyzer — github.com. <https://github.com/s2e-lab/hf-model-analyzer.git> (2025), [Accessed 11-11-2025]
23. Hub client library — huggingface.co. https://huggingface.co/docs/huggingface_hub/index (2025), [Accessed 07-11-2025]
24. Hugging Face – The AI community building the future. <https://huggingface.co/> (2025), [Accessed 07-11-2025]
25. huggingface/transformers — github. https://github.com/huggingface/transformers/blob/main/src/transformers/modeling_utils.py (2025), [Accessed 13-10-2025]
26. Kaggle: Your Machine Learning and Data Science Community. <https://www.kaggle.com/> (2025), [Accessed 07-11-2025]
27. Malicious ML models discovered on Hugging Face platform | ReversingLabs. <https://www.reversinglabs.com/blog/r1-identifies-malware-ml-model-hosted-on-hugging-face> (2025), [Accessed 09-11-2025]
28. Meta releases llamafirewall, an open-source defense against ai hijacking — deeplearning.ai. <https://www.deeplearning.ai/the-batch/meta-releases-llamafirewall-an-open-source-defense-against-ai-hijacking/> (2025), [Accessed 12-11-2025]
29. ModelScope. <https://modelscope.cn/home> (2025), [Accessed 07-11-2025]
30. NumPy documentation; NumPy v2.3 Manual — numpy.org. <https://numpy.org/doc/stable/> (2025), [Accessed 07-11-2025]
31. OpenCSG. <https://opencsg.com/> (2025), [Accessed 07-11-2025]
32. pandas documentation; pandas 2.3.3 documentation — pandas.pydata.org. <https://pandas.pydata.org/docs/> (2025), [Accessed 07-11-2025]
33. Python client libraries | Google Cloud Documentation. <https://docs.cloud.google.com/python/docs/reference/storage/latest> (2025), [Accessed 11-11-2025]
34. PyTorch documentation; PyTorch 2.9 documentation — docs.pytorch.org. <https://docs.pytorch.org/docs/stable/index.html> (2025), [Accessed 07-11-2025]
35. pytorch/Phi-4-mini-instruct-FP8 at main — huggingface.co. <https://huggingface.co/pytorch/Phi-4-mini-instruct-FP8/tree/main> (2025), [Accessed 10-11-2025]
36. Spark NLP. <https://sparknlp.org/> (2025), [Accessed 07-11-2025]
37. strace. <https://github.com/strace/strace> (2025), [Accessed 12-10-2025]
38. syscalls(2) - Linux manual page — man7.org. <https://man7.org/linux/man-pages/man2/syscalls.2.html#:~:text=/usr/include/asm/unistd.h> (2025), [Accessed 30-10-2025]
39. Tcpdump; libpcap. <https://www.tcpdump.org/> (2025), [Accessed 12-10-2025]
40. User Guide — scikit-learn.org. https://scikit-learn.org/stable/user_guide.html (2025), [Accessed 07-11-2025]
41. What is a Container? | Docker — docker.com. <https://www.docker.com/resources/what-container/> (2025), [Accessed 11-11-2025]
42. What is few shot prompting? <https://www.ibm.com/think/topics/few-shot-prompting> (2025), [Accessed 11-11-2025]
43. Dynahug-detector-artifact. <https://dynahug-detector.github.io/> (2026), [Accessed 15-04-2026]
44. Slutterprime — pypi.org. <https://pypi.org/project/slutterprime/> (2026), [Accessed 28-01-2026]

45. 0dayCTF: Online - reverse shell generator. <https://www.revshells.com/> (2021), [Accessed 09-04-2026]
46. AI, O.: Introducing GPT-5.2 — openai.com. <https://openai.com/index/introducing-gpt-5-2/> (2025), [Accessed 29-01-2026]
47. AI, P.: Modelscan: Protection against model serialization attacks. <https://github.com/protectai/modelscan> (2025), gitHub repository, accessed: 2025-10-11
48. of Bits, T.: Fickling: A python pickling decompiler and static analyzer. <https://github.com/trailofbits/fickling> (2025), gitHub repository, accessed: 2025-10-11
49. Brown, P., Brown, A., Gupta, M., Abdelsalam, M.: Online malware classification with system-wide system calls in cloud iaas. In: 2022 IEEE 23rd International Conference on Information Reuse and Integration for Data Science (IRI). pp. 146–151. IEEE (2022)
50. Canonical: Strace man page, <https://manpages.ubuntu.com/manpages/jammy/man1/strace>
51. Casey, B., Santos, J.C.S., Mirakhorli, M.: A large-scale exploit instrumentation study of ai/ml supply chain attacks in hugging face models (2024), <https://arxiv.org/abs/2410.04490>
52. Cohen, D.: Data scientists targeted by malicious hugging face ml models with silent backdoor. <https://jfrog.com/blog/data-scientists-targeted-by-malicious-hugging-face-ml-models-with-silent-backdoor/> (2024), [Accessed 27-10-2025]
53. Contributors, P.: weights_only_unpickler.py – pytorch. https://github.com/pytorch/pytorch/blob/main/torch/_weights_only_unpickler.py (2025), gitHub repository, Accessed: 2025-11-10
54. Developers, P.: CPython: The python programming language. <https://github.com/python/cpython>, gitHub repository, accessed 2025-10-12
55. Docker, I.: Docker: Accelerated Container Application Development — docker.com. <https://www.docker.com/> (2025), [Accessed 06-11-2025]
56. Face, H.: Pickle scanning (hub documentation). <https://huggingface.co/docs/hub/en/security-pickle#what-we-have-now> (2025), hugging Face documentation, accessed: 2025-10-11
57. Face, H.: Third-party scanner: Jfrog. <https://huggingface.co/docs/hub/en/security-jfrog> (2025), hugging Face documentation, accessed: 2025-10-11
58. Face, H.: Third-party scanner: Protect ai. <https://huggingface.co/docs/hub/en/security-protectai> (2025), hugging Face documentation, accessed: 2025-10-11
59. George, D., Mauldin, A., Mitchell, J., Mohammed, S., Slater, R.: Static malware family clustering via structural and functional characteristics. *SMU Data Science Review* **7**(2), 4 (2023)
60. Gorla, A., Tavecchia, I., Gross, F., Zeller, A.: Checking app behavior against app descriptions. In: Proceedings of the 36th international conference on software engineering. pp. 1025–1035 (2014)
61. Hijazi, F.: “totally-harmless-model” – model card. <https://huggingface.co/FarisHijazi/totally-harmless-model> (2025), accessed: 2025-11-10
62. HuggingFace: State of open source on hugging face: Spring 2026 — huggingface.co. <https://huggingface.co/blog/huggingface/state-of-os-hf-spring-2026> (2026), [Accessed 09-04-2026]
63. Kassianik, P., Saglam, B., Chen, A., Nelson, B., Vellore, A., Aufiero, M., Burch, F., Kedia, D., Zohary, A., Weerawardhena, S., et al.: Llama-3.1-foundationai-securityllm-base-8b technical report. arXiv e-prints pp. arXiv-2504 (2025)

64. Kellas, A.D., Christou, N., Jiang, W., Li, P., Simon, L., David, Y., Kemerlis, V.P., Davis, J.C., Yang, J.: Pickleball: Secure deserialization of pickle-based machine learning models (extended report) (2025), <https://arxiv.org/abs/2508.15987>
65. Liu, Y., Cao, J., Liu, C., Ding, K., Jin, L.: Datasets for large language models: A comprehensive survey (2024), <https://arxiv.org/abs/2402.18041>
66. Lundberg, S.M., Lee, S.I.: A unified approach to interpreting model predictions. Curran Associates, Inc. (2017), <http://papers.nips.cc/paper/7062-a-unified-approach-to-interpreting-model-predictions.pdf>
67. Meta: meta-llama/Llama-3.1-8B-Instruct. <https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct> (2025), [Accessed 09-11-2025]
68. mmaitre314: picklescan: Security scanner detecting python pickle files performing suspicious actions. <https://github.com/mmaitre314/picklescan> (2025), gitHub repository, accessed: 2025-10-11
69. Montalbano, E.: (<https://www.darkreading.com/application-security/hugging-face-ai-platform-100-malicious-code-execution-models>) (Feb 2024)
70. News, T.H.: New Hugging Face Vulnerability Exposes AI Models to Supply Chain Attacks. <https://thehackernews.com/2024/02/new-hugging-face-vulnerability-exposes.html> (2024), [Accessed 09-11-2025]
71. News, T.H.: Over 100 Malicious AI/ML Models Found on Hugging Face Platform. <https://thehackernews.com/2024/03/over-100-malicious-aiml-models-found-on.html> (2024), [Accessed 09-11-2025]
72. News, T.H.: Malicious ML Models on Hugging Face Leverage Broken Pickle Format to Evade Detection. <https://thehackernews.com/2025/02/malicious-ml-models-found-on-hugging.html> (2025), [Accessed 09-11-2025]
73. Nmap Project: Ncat — nmap's netcat replacement. <https://nmap.org/ncat/> (2025), accessed: 2025-11-10
74. PACULL, F.: Loading data into a Pandas DataFrame — a performance study. https://www.architecture-performance.fr/ap_blog/loading-data-into-a-pandas-dataframe-a-performance-study/ (2019), [Accessed 14-10-2025]
75. Perry, C.: Colab Nvidia GPU. <https://blog.tensorflow.org/2022/09/colabs-pay-as-you-go-offers-more-access-to-powerful-nvidia-compute-for-machine-learning.html> (2022), [Accessed 13-11-2025]
76. Pimenta, T.S.R., Ceschin, F., Gregio, A.: Androidgyny: Reviewing clustering techniques for android malware family classification. Digital Threats **5**(1) (Mar 2024). <https://doi.org/10.1145/3587471>, <https://doi.org/10.1145/3587471>
77. Poireault, K.: Malicious AI Models on Hugging Face Exploit Novel Attack Technique. <https://www.infosecurity-magazine.com/news/malicious-ai-models-hugging-face/> (2025), [Accessed 09-11-2025]
78. Prusa, J., Khoshgoftaar, T.M., Seliya, N.: The effect of dataset size on training tweet sentiment classifiers. In: 2015 IEEE 14th International Conference on Machine Learning and Applications (ICMLA). pp. 96–102 (2015). <https://doi.org/10.1109/ICMLA.2015.22>
79. PyPI: PyPI &xB7; The Python Package Index — pypi.org. <https://pypi.org> (2025), [Accessed 10-11-2025]

80. Rajput, D., Wang, W.J., Chen, C.C.: Evaluation of a decided sample size in machine learning applications. *BMC Bioinformatics* **24**(1), 48 (Feb 2023)
81. Ramezan, C.A., Warner, T.A., Maxwell, A.E., Price, B.S.: Effects of training set size on supervised machine-learning land-cover classification of large-area high-resolution remotely sensed data. *Remote Sens. (Basel)* **13**(3), 368 (Jan 2021)
82. Rondanini, C., Carminati, B., Ferrari, E., Kundu, A., Gaudiano, A.: Malware detection at the edge with lightweight llms: A performance evaluation. *ACM Transactions on Internet Technology*
83. Ronik: (2024), <https://weam.ai/blog/guide/huggingface-statistics/>
84. Senanayake, J., Kalutarage, H., Al-Kadri, M.O.: Android mobile malware detection using machine learning: A systematic review. *Electronics* **10**(13) (2021). <https://doi.org/10.3390/electronics10131606>, <https://www.mdpi.com/2079-9292/10/13/1606>
85. swisskyrepo: Github - swisskyrepo/payloadsallthethings: A list of useful payloads and bypass for web application security and pentest/ctf. <https://github.com/swisskyrepo/PayloadsAllTheThings> (2019), [Accessed 09-04-2026]
86. Ucci, D., Aniello, L., Baldoni, R.: Survey of machine learning techniques for malware analysis. *Computers & Security* **81**, 123–147 (2019). <https://doi.org/https://doi.org/10.1016/j.cose.2018.11.001>, <https://www.sciencedirect.com/science/article/pii/S0167404818303808>
87. Zhao, J., Wang, S., Zhao, Y., Hou, X., Wang, K., Gao, P., Zhang, Y., Wei, C., Wang, H.: Models are codes: Towards measuring malicious code poisoning attacks on pre-trained model hubs. In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. p. 2087–2098. ASE '24, ACM (Oct 2024). <https://doi.org/10.1145/3691620.3695271>, <http://dx.doi.org/10.1145/3691620.3695271>
88. Zollllll: “dont_download_this” – model card. https://huggingface.co/Zollllll/dont_download_this (2025), accessed: 2025-11-10

9 Appendix

Table 12: Number of PTMs on Top six (6) Model Hubs.

Hub	#Models	Formats	Text Generation	Text Classification	Feature Extraction
Hugging Face [24]	1881296	Pytorch, Tflow, gguf	296411	98414	13864
GitHub [21]	150685	Pytorch, Tflow, gguf	1732	4427	1957
Spark NLP [36]	135372	Custom	25181	35248	N/A
OpenCSG [31]	111189	Pytorch, Tflow, gguf	4224	278	484
ModelScope [29]	80574	Pytorch, Tflow, gguf	20307	1172	740
Kaggle [26]	3140	Pytorch, Tflow, gguf	800	197	N/A

Example 1.1: MALHUG payload [13]

```
2: c GLOBAL '___builtin___
eval'
20: q BINPUT 0
22: X BINUNICODE
'exec('\''\''\nprint("Hello, I am
'Gorden
Ramsey!")\n\''\'' or dict()'
```

Example 1.2: PyPI payload

```
2: c GLOBAL 'raft run'
12: q BINPUT 0
14: X BINUNICODE "zsh -c
'zmodload
zsh/net/tcp
&& ztcp 127.0.0.1 4444
&& zsh >\&$REPLY 2>\&$REPLY
0>\&$REPLY'"
```

9.1 DYNAHUG Overview

DYNAHUG’s steps are color mapped in the algorithm (Figure 6) and the workflow diagram (Figure 1): *Crawling* steps are highlighted in orange, *Dynamic Analysis* steps in blue and *Model Training* steps in green.

The goal of DYNAHUG is to learn the benign behavior of PTMs for a specific ML task (e.g., text generation). It achieves this via dynamic analysis and machine learning. Given a PTM under test (PUT) and the task tag of the PTM (e.g., text generation), DYNAHUG learns the benign behavior of benign models in the task tag and identifies the PUT to be either *malicious* or *benign*.

Fig. 6: DYNAHUG Algorithm

```

Input: TAG, modelToAnalyze
Output: Malicious or Benign
1: // Phase 1: Crawling
2:  $M_{tag} \leftarrow \text{fetchModels}(\text{"PyTorch"}, TAG)$ 
3: // Phase 2: Dynamic Analysis
4:  $traces \leftarrow \emptyset$ 
5: for  $M_i \in M_{tag}$  do Model deserialization
6:    $\text{executeModelInSandbox}(M_i)$ 
7:    $traces \leftarrow traces \cup \text{getTraces}(M_i)$ 
8: end for
9: // Phase 3: Model Training
10:  $X \leftarrow \emptyset$   $\triangleright$  Training dataset
11: for  $trace_i \in traces$  do
12:    $d_i \leftarrow \text{dataProcessing}(trace_i)$ 
13:    $X \leftarrow X \cup \text{featureEngineering}(d_i)$ 
14: end for
15:  $\mathcal{M} \leftarrow \emptyset$   $\triangleright$  List of classifiers to evaluate
16:  $\text{modelArch} \leftarrow \text{OneClassSVM}$ 
17:    $\text{hyperparams} \leftarrow \{kernel : [RBF, linear, \dots], \dots\}$ 
18: for  $h \in \text{hyperparams}$  do  $\triangleright$  Grid Search CV
19:    $\mathcal{M} \leftarrow \mathcal{M} \cup \text{train}(X, \text{modelArch}, h)$ 
20: end for
21:  $(m_{best}, h_{best}) \leftarrow \text{evaluation}(\mathcal{M})$ 

```

To achieve this, DYNAHUG first fetches benign PTMs for the task at hand from a Model Hub (e.g., Hugging Face) by filtering for the task tag and the model format (e.g, Pickle) (Figure 6:line 2). Once a large number (2K) of benign PTMs are retrieved, DYNAHUG loads (deserialises) each model in a sandbox and collects its system call traces using runtime observability tools like strace (Figure 6:line 5,6). It then processes the collected traces into meaningful features, transforming them into a structured and interpretable format (Figure 6:line 11). Once the traces are processed, it engineers new features from the existing data, i.e., binary and frequency features (Figure 6:line 12). Next, DYNAHUG learns an ML model detector by first tuning the model to maximize performance by passing this processed data for training alongside the hyperparameters and ML architectures (Figure 6:line 15,16,19). Using the F1-score as the main evaluation metric, the best performing model is then chosen as DYNAHUG. Finally,

at inference, DYNAHUG detects whether the *PUT* is benign or malicious. It first collects and processes the trace data of the PUT (similarly to the *Model training* step), then passes the processed data for a detection output of *malicious* or *benign*.

9.2 Dataset collection and crawling setup

DYNAHUG utilizes functions from the `huggingface_hub` [23] library to fetch a list of repository names from HF to crawl. We pass *likes* and *pipeline tag* as arguments to this function to obtain a list of repositories ranked in descending order of *likes* and clustered based on the *pipeline tag* in HF. By iterating through

Table 13: DYNAHUG’s performance of sequence features on varying n-grams

	Benign HF (2025)	Malicious Real (25)	Malicious Injected (2000)	TP	TN	FP	FN	Precision	Recall	F1-score
DYNAHUG (2-gram)	51	24	2000	2024	1974	51	1	0.9754	0.9995	0.9873
3-gram	36	24	1922	1946	1989	36	79	0.9818	0.9610	0.9713
4-gram	27	24	1901	1925	1998	27	100	0.9861	0.9506	0.9681

each repository in the list, we obtain metadata information such as the type of files being stored. We look through the list of files at the top level of the repository and check for the presence of `pytorch_model.bin`. For each PyTorch model file, we check its HF security status and skip it if the model file was identified by HF as “unsafe”. We retrieve the size of the file and estimate the memory usage when it is deserialized during dynamic analysis to avoid Out Of Memory (OOM) issues. If the memory safety check succeeds, the `pytorch_model.bin` file is downloaded from the repository and we check that it contains a Pickle file.³ If all the aforementioned conditions are met, we store any relevant metadata about the repository in a CSV file, i.e., number of likes, downloads, date of last commit and HF security tool [56–58] detection.

9.3 Baseline Selection

Security Scanners: We chose PickleScan [68] and Modelscan [47] as they are the closest open-source alternatives to the currently used scanners on Hugging Face [56, 58]. PickleScan is the base version of Hugging Face’s HF PickleScan, while Modelscan is developed by the same company, ProtectAI, which also developed Hugging Face’s Guardian. We also use Fickling [48], developed by Trail of Bits, as it provides novel detection methods. To the best of our knowledge, ModelTracer [51] is the only open-source dynamic scanner for malicious PTM detection.

LLM baseline: We employ Llama-3.1-8B-Instruct-tuned (Llama-3.1) for the LLM baseline since the Meta-Llama series of LLMs is widely recognized as one of the state-of-the-art open source models and it has demonstrated strong capabilities in security-related applications [28, 63, 82]. It is free to use, computationally efficient and instruction-tuned to produce standardized outputs. GPT-5.2 was also used as a baseline due to its strong performance amongst the current state-of-the-art LLMs.

Excluded Scanners: For **RQ2**, we exclude the following types of scanners because they are not applicable to our threat model, impractical in practice, or impossible to execute on our dataset:

a.) Hugging Face Scanners: We note that Hugging Face (HF) runs closed-source versions of security scanners on uploaded models. However, we do not evaluate against the HF scanners as we already use them to filter for models that are

³ This step is required since `.bin` files do not guarantee the presence of `.pickle` or `.pkl` files.

benign and thus would be inherently biased. We also cannot feasibly evaluate the HF Scanners against our injected malicious set as the only way to run them would be to upload them to HF –calling for the need of upload of 6000 injected malicious models– which comes with a risk of violating the platform’s policies.

b.) Restricted Loading Environments (RLEs): We determined that RLEs were not applicable to our threat model, as they require a security-aware user. Specifically, we note that Weights-Only Unpickler [53], developed by PyTorch, can be bypassed by merely instructing the user to set `weights_only=False` while loading the model. PickleBall [64] is also a RLE that overrides PyTorch’s Weights-Only Unpickler [53]. Furthermore, PickleBall requires manual specification of security policies for each model, which requires significant time and effort. It does not scale to our setting where we scan thousands of PTMs (over 25K) or generalise to model hub security scenarios (e.g., Hugging Face containing 2.2M models [62]).

9.4 Baseline setting

We compare DYNAHUG to the open-source state-of-the-art (SOTA) malicious PTM detectors. In particular, three static detectors (PickleScan [68], Modelscan [47], Fickling [48]) and one dynamic detector (ModelTracer [51]). We also compare DYNAHUG to open-weights and closed-source SOTA LLMs, namely Llama-3.1 [67] and GPT-5.2 [46].

Baseline Scanner setup: We execute PickleScan and ModelScan via a Python script that calls a shell command to run the tool, given the path to the PUT. We employ `analyze_pickle_safety`, to obtain various security threat levels (such as `LIKELY_UNSAFE`, `OVERTLY_MALICIOUS`) for Fickling. To run ModelTracer, we execute the code provided by its GitHub repository [22].

LLM baseline setup: For the LLM baseline evaluation, GPT-5.2 and Llama-3.1 were provided raw traces obtained from dynamic analysis as its input. This design choice was made since LLMs are known to have a strong capability to develop a rich semantic understanding of text, enabling it to have a deeper insight into the relationship between the different system calls which occur within an `strace` [37] log file. Due to the small context window of Llama-3.1-8B-Instruct-tuned [67] (i.e., maximum 128k tokens [14]) and the limited CUDA memory of the NVIDIA A100 chip (i.e., 64GB CUDA memory [75]), passing in the `strace` logs in its entirety would leave the program susceptible to either CUDA OOM error or the LLM losing out on vital information for accurate detections. To mitigate this, we filter out which system calls are needed to be passed for the LLMs to have an understanding of the overall functioning of the deserialized PyTorch file. We curated a list of 35 system calls borrowed from *Brown et al.* [49] alongwith an additional 23 system calls to filter the `strace` logs. The same filtered `strace` logs were provided to GPT-5.2 to maintain similar settings with Llama-3.1. The LLMs was instructed with Few-Shot prompting [42] to use this filtered `strace` log to look for any red flags during pickle deserialization. Figure 8 portrays the system prompt and a sample LLM response. To mitigate randomness, the temperature of the LLMs was set to zero (0).

9.5 Ablation, Probing and Sensitivity Setting

Feature set: All feature sets displayed in Figure 3 were used to study the effect of each feature on the model performance. In this study, the model complexity was varied while keeping the rest of the hyperparameter settings the same as the DYNAHUG (default). The *presence* only model was trained on features representing the presence of individual system calls. The *frequency* only model was trained on frequency of individual system calls. The *presence and frequency* model (DYNAHUG (default)) was trained on a combination of *presence* and *frequency* features. Subsequently, *Sequence* and *Process Sequence* features were concatenated to train the respective *presence*, *frequency*, *sequence* and *presence, frequency, sequence, process sequence* models.

Architecture: *OCSVM*, *SGDSVM* and *IF* were trained with their best hyperparameter setting.

Analysis Type: The *dynamic* model was trained with the workflow showcased in Figure 1. The *static* model processed static opcodes from disassembled models instead of `strace` log data while following the same workflow as the *dynamic* model. The *hybrid* model was trained with the combined feature set of *static* and *dynamic* models.

Clustering: To test whether clustering improves performance, we trained a model without task-based filtering. We then compare this model against DYNAHUG (default).

Dataset Size Sensitivity: For sensitivity analysis (dataset sizes), we trained our model on the data retrieved from the top 1000, 2000 (DYNAHUG (default)) and 3000 repositories, sorted by likes.

9.6 Additional Experiments

Advanced Attacks: We investigated the effectiveness of DYNAHUG and four (4) baseline scanners when faced with advanced malicious attacks. We focus on attacks that employ four (4) stealthy techniques, namely, (a) debugging and Anti-VM detection, (b) staged payloads, (c) obfuscated payloads and (d) delayed execution. We orchestrated all four attacks using the hard-to-detect PyPI library (slutterprime [44]), resulting in four advanced attack variants.

We found that DYNAHUG scales to these attacks, it detects these malicious PTMs containing such payloads since they still use system calls that are uncommon in benign PTMs. Among the four (4) SOTA scanners, only Fickling and ModelTracer detected these attacks. However, ModelTracer fails to detect the Anti-VM attack. Fickling detects the advanced malicious PTM because of the presence of the PyPI library import (which is not in its whitelist) and ModelTracer detects them because their traces still exhibit blacklisted system calls. ModelTracer does not detect the Anti-VM attack as it does not have `read()` and `openat()` as blacklisted syscalls (which are used to detect whether a VM is being used). DYNAHUG detects the Anti-VM attack because it recognises the additional `read()` and `openat()` calls. We note that even though Fickling

and ModelTracer detected these attack, their detection methods lead to a lot of false positives or false negatives for unseen PTMs, due to their non-exhaustive rule lists. This result demonstrates the scalability of DYNAHUG to advanced attacks and the effectiveness of dynamic analysis in exposing such attacks. Our artifact [43] contains the malicious models resulting from this analysis, and our results.

DYNAHUG scales to advanced attacks. It outperforms the SOTA scanners in detecting malicious PTMs invoking Anti-VM detection, as well as staged, obfuscated and delayed payloads.

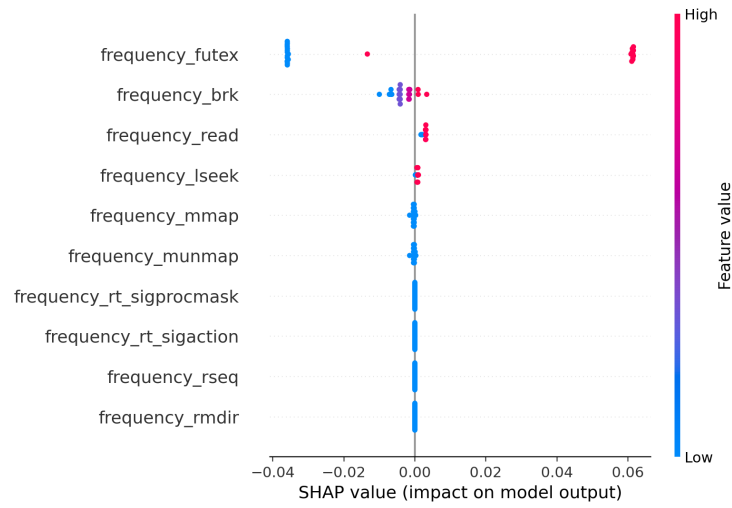


Fig. 7: Top-10 important features that explain DYNAHUG's predictions on benign models

Fig. 8: System prompt and LLM response

```

Prompt:
You are analyzing strace logs to detect malicious
activity in PyTorch files.

Look for RED FLAGS like these:
- Process creation (execve, fork, clone, ...)
- Network connections (socket, connect, bind, ...)
- File writes outside normal directories
- Access to sensitive files (/etc/passwd, SSH keys, ...)
- Essentially unusual system calls
during deserialization

IMPORTANT: You must respond in EXACTLY this format:
Answer: Malicious
Reason: Brief explanation
...
Example 1:
Log shows: execve("/bin/bash", ["/bin/bash", "-c", "rm
-rf /"])
Answer: Malicious
Reason: Executes destructive shell command during
deserialization
...
-----
LLM Response:
Answer: Malicious
Reason: The process attempts to connect to a suspicious
IP address (192.248.1.167) on port 4242, which could
be a sign of a malicious activity.

```